

Rust enum hint sheet

Valid Parentheses with direct conversion

Rewrite your stack solution using Kind and Bracket. Assume the input contains only ()[]{}. Convert each character directly into a Bracket, then match on its direction.

1 Represent kind and direction

```
#[derive(PartialEq)]
enum Kind {
    Round,
    Square,
    Curly,
}
enum Bracket {
    Open(Kind),
    Close(Kind),
}
use Bracket::*;
use Kind::*;
```

An enum value is one of its named variants. Open(Kind) and Close(Kind) each carry a kind. PartialEq enables == and !=. The use lines let you write Open(Round) instead of Bracket::Open(Kind::Round).

2 Create a value and take it apart

```
let b = Close(Square);
match b {
    Open(kind) => println!("open: {}", kind == Square),
    Close(kind) => println!("close: {}", kind == Square),
}
```

This prints close: true. In Close(kind), the pattern checks the direction and gives the stored value the name kind. Kind is a type; kind is a variable. You could name that variable k instead.

3 Convert a character

Finish the four similar arms. The return type is Bracket, so each arm produces an opening or closing bracket directly.

```
fn to_bracket(c: char) -> Bracket {
    match c {
        '(' => Open(Round),
        ')' => Close(Round),
        '[' => todo!("opening square"),
        ']' => todo!("closing square"),
        '{' => todo!("opening curly"),
        '}' => todo!("closing curly"),
        _ => unreachable!(),
    }
}
```

_ matches any remaining character. unreachable!() panics if the input violates our six-character assumption. An invalid bracket order such as ([)] still uses allowed characters and must return false.

todo!() marks unfinished code: it compiles, but panics if reached. Replace each one before testing that path.

Finish your stack solution

Put the enums, use lines, and to_bracket above impl Solution. On LeetCode, keep its supplied Solution wrapper. Fill in A, B, and C.

```
impl Solution {
    pub fn is_valid(s: String) -> bool {
        let mut stack: Vec<Kind> = vec![];
        for c in s.chars() {
            match to_bracket(c) {
                Open(kind) => {
                    todo!("A: remember the opening kind");
                }
                Close(kind) => {
                    todo!("B: check the latest opening kind");
                }
            }
        }
        todo!("C: did every opening get closed?")
    }
}
```

4 Hints for A and C

For A, use `stack.push(kind)`. `Vec<Kind>` stores unmatched opening kinds; after A is filled, you may omit this type annotation. For C, use `stack.is_empty()`. Leave off the final semicolon to return its `bool`.

5 Two choices for the closing arm

For B, choose ONE snippet below. Each removes at most one opening kind. After a match, continue the loop; do not return `true` early.

Choice 1 Compare the result of popping

```
if stack.pop() != Some(kind) {
    return false;
}
```

`pop()` removes and returns `Some(value)`, or returns `None` when empty. An empty stack or a different kind makes `!= true`, so the input is rejected.

Choice 2 Check for an empty stack first

```
if stack.is_empty() || stack.pop().unwrap() != kind {
    return false;
}
```

`unwrap()` extracts the popped kind; it would panic on an empty result. This guard is safe because `||` stops when the left side is true. Keep the empty check first and use `||`, not `|`.

6 Check different failure cases

Input	Expected	What it checks
"()[]{}"	true	Separate pairs
"{[]}"	true	Nested pairs
"()"	false	Wrong kind
"([)]"	false	Wrong closing order
")" or "())"	false	Closing without an available opening
"(("	false	Openings left at the end